

DBSCAN Clustering in R: Density-Based Methods and Model Selection

cluster_dbscan – DBSCAN

DBSCAN identifies **dense regions** separated by sparse areas, enabling the detection of noise and arbitrarily shaped clusters — a capability that partition-based methods such as k-means and PAM fundamentally lack. The model below is configured with `minPts = 3`.

How to interpret the visualization below: each colored region represents an arbitrarily shaped cluster detected automatically by DBSCAN — not constrained to spherical or convex forms. Points shown in **gray** are classified as **noise** — isolated observations that do not belong to any dense cluster. Unlike k-means or k-medoids, DBSCAN **does not require the number of clusters to be specified in advance**: the algorithm autonomously discovers how many dense regions exist in the data.

Cluster Distribution (iris)

clu:	0	1	2	3	4
	26	47	38	4	35

Cluster **0** represents noise points (26 observations flagged as outliers not belonging to any dense region).

External Evaluation

Clustering entropy: **0.3037** — the lowest among all methods evaluated, reflecting greater cluster cohesion. Clusters 1, 2, and 3 achieve zero entropy, indicating near-perfect alignment with the ground-truth species labels.

Reference: Ester et al. (1996) — A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.

DBSCAN – Theoretical Foundations

DBSCAN was proposed to overcome the limitations of partition-based methods (k-means, PAM), which assume convex cluster shapes and require k to be specified in advance. DBSCAN discovers clusters as **dense regions separated by sparse areas**, without any prior assumption about the number of clusters.

Point Classification

Core Point

Has at least `minPts` neighbors within radius `eps`.
Defines the interior of a dense region.

Border Point

Within `eps` of a core point, but has fewer than `minPts` neighbors itself. Lies on the edge of a cluster.

Noise Point

Neither a core nor a border point. Treated as an outlier and assigned to cluster 0.

Key Parameters

`eps` (epsilon)

The neighborhood radius. Two points are neighbors if their distance is at most `eps`. Smaller `eps` = stricter density requirement.

`minPts`

Minimum number of points within `eps` to qualify as a core point. Higher `minPts` = denser clusters required. A common heuristic: set `minPts` = $2 \times \text{dimensions}$, then use the k-distance graph to choose `eps`.

Advantages and Limitations

Advantages

Discovers clusters of **arbitrary shape**; does not require k to be specified; naturally identifies noise and outliers; robust to non-convex geometries.

Limitations

Sensitive to `eps` and `minPts` choices; struggles with clusters of **varying density**; performance degrades in high-dimensional spaces (curse of dimensionality).

 **Reference:** Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pp. 226–231. AAAI Press.

Hyperparameter Optimization in Clustering – Theoretical Foundations

Hyperparameters define the behavior of a clustering algorithm before training begins. Unlike supervised learning, there is no target variable to guide selection — making this a fundamentally harder optimization problem where internal structure must serve as the sole guide.

Core Concepts

What are hyperparameters?

Settings defined before training (e.g., `k` in K-means, `eps` and `minPts` in DBSCAN). They shape the structure of the solution space and determine the granularity and geometry of discovered clusters.

The unsupervised challenge

Without a target variable, there is no direct loss function to minimize. Internal metrics — cohesion and separation — must serve as proxies for clustering quality, making evaluation inherently indirect.

Search strategies

Grid search, random search, and curve-based methods (elbow, silhouette) are common approaches. Automated tools like `clu_tune` systematize and streamline this selection process.

Evaluation Criteria

The following internal metrics guide hyperparameter selection in the absence of ground truth labels:

Clustering Entropy

When class labels are available for posterior evaluation, entropy measures the purity of clusters relative to true classes — lower entropy indicates more homogeneous groupings.

Silhouette Score

Measures how similar a point is to its own cluster versus neighboring clusters. Ranges from -1 to 1, where higher values indicate better-defined, well-separated clusters.

Elbow Method / Kneedle Algorithm

Identifies the inflection point in a quality curve as `k` increases. The Kneedle algorithm formalizes this by detecting the point of maximum curvature programmatically.

clu_tune – Automated Hyperparameter Tuning

Building on the clustering methods explored so far, a critical challenge in unsupervised learning is selecting the right hyperparameters — such as **k** — which directly impact cluster quality. The `clu_tune` function addresses this by **systematically testing different parameter values** and selecting the best configuration based on internal evaluation criteria, without relying on external labels.

How It Works

Defines a search space (ranges) and evaluates each configuration using internal metrics (e.g., clustering entropy). The configuration with the lowest entropy is automatically selected.

```
# install.packages("daltoolbox")
library(daltoolbox)

data(iris)

# Define the search space: k from 1 to 10
# k=0 is a placeholder — the actual value
# will be determined automatically by clu_tune
model <- clu_tune(cluster_kmeans(k=0),
  ranges = list(k = 1:10))

# fit() evaluates each k in the search space
# and selects the best configuration internally
model <- fit(model, iris[,1:4])
```

Key Advantage

Eliminates manual selection of k, making the process fully reproducible and systematic. Reduces bias introduced by trial-and-error experimentation.

```
# Best k found automatically
model$k
## [1] 9

# Apply the model with the selected k
clu <- cluster(model, iris[,1:4])
table(clu)
##  1  2  3  4  5  6  7  8  9
## 12  9 50 12 20  4 17 14 12

# Evaluate using true labels (iris$Species)
eval <- evaluate(model, clu, iris$Species)
## $clustering_entropy
## [1] 0.1143797
```

Important Caveat

The best internally selected k does not always reflect the true underlying structure of the data. Interpretation of the resulting clusters remains essential.

Results with k = 9

- The algorithm selected **k = 9** as the optimal configuration after scanning values from 1 to 10
- Clustering entropy drops to **0.114** — yielding far purer groups than any fixed-k approach
- **7 out of 9 clusters** achieve zero entropy — indicating near-perfect class separation
- Interpretation remains necessary: smaller clusters may represent genuine subgroups or signs of overfitting

Method Comparison

The table below summarizes the results obtained with each clustering approach applied to the same dataset. **Entropy** is the external evaluation metric — the **lower** the value, the better the alignment between the formed clusters and the true class labels. Methods are ordered from worst to best performance.

Method	Entropy ↓	Clusters
K-means (<i>no normalization</i>)	0.7375	3
K-means (<i>min-max</i>)	0.4178	3
PAM	0.3939	3
DBSCAN	0.3037	4 + noise
clu_tune (<i>k=9</i>)	0.1144	9

↓ Lower entropy = better separation between clusters and true class labels

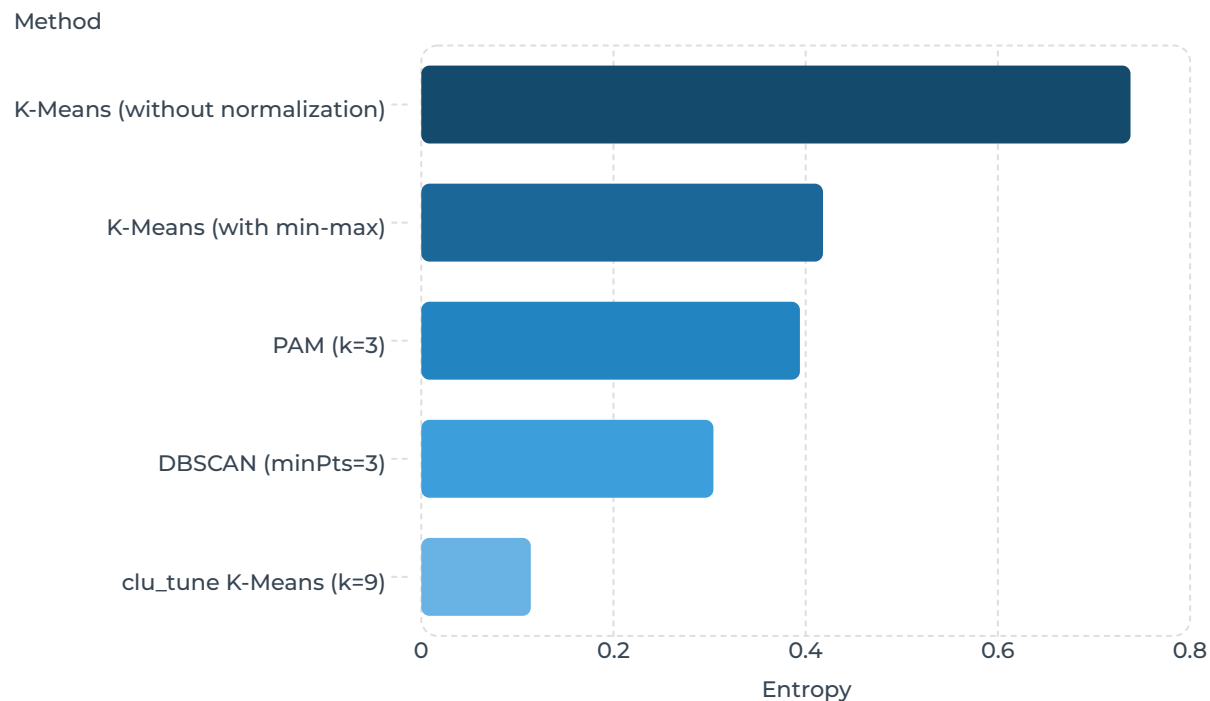
Key Takeaways

- **Normalization matters:** applying min-max scaling to K-means reduced entropy from 0.74 to 0.42
- **PAM is robust:** outperforms raw K-means without requiring additional preprocessing
- **DBSCAN is flexible:** does not require a predefined `k`, but may produce noise points
- **Hyperparameter search wins:** `clu_tune` achieves the lowest entropy, at the cost of more clusters

📌 **Key concept:** Evaluation with `evaluate()` is strictly pedagogical — the true class labels **never participate in training**, only in the posterior interpretation of results.

Comparative Analysis: Clustering Entropy

What is clustering entropy? Entropy measures the **purity of clusters** relative to the actual external labels (Species). A value of **0 indicates perfectly pure clusters** — each cluster contains only one species — while higher values indicate greater mixing of species within clusters. The lower the entropy, the more cohesive and interpretable the resulting groupings.



What do these results mean in practice? Normalization alone reduces K-Means entropy from 0.74 to 0.42 — a substantial gain from preprocessing alone. PAM outperforms raw K-Means without requiring normalization, making it a more robust choice when preprocessing options are limited. DBSCAN achieves strong separation without predefining k , but at the cost of generating noise points. The `clu_tune` approach reaches the lowest entropy (0.11), yet does so with $k=9$ — far above the 3 actual species. This highlights an important risk: **mechanically optimizing a quality indicator is no substitute for sound analytical judgment**. The best method always depends on balancing purity, interpretability, and alignment with the study's objectives.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

